Spireslayer

Advanced Command-Line Game Interpreter Design & Implementation

CMPE 230: Systems Programming – Spring 2026

Yiğit Sarp Avcı 2023400048
Doğukan Sungu 2023400210

Abstract

This document outlines the architectural and implementation specifics of `spireslayer`, a custom-built command-line interpreter in C11. The application accurately mirrors a simplified, deterministic subset of the game *Slay the Spire*. We detail the program's strict lexical tokenization strategies, a dynamic backtracking syntax parser, and an aggressively optimized virtual state-management module. The subsequent sections illustrate how the application ensures strict syntactic adherence, zero memory leak trajectories under extreme stress tests, and scalable data structure deployments optimized for asymptotic lookup efficiency.

Contents

1 Introduction and Architectural Scope **2**

2 Advanced Lexical Validation **2**
2.1 Raw Stream Processing . . . 2
2.2 Tokenization and Spatial Geometry Tracking . . . 2

3 The Backtracking Syntactic Evaluator **2**
3.1 Greedy Matching Strategy . . . 3
3.2 Semantic Keyword Blacklisting and Contiguity Enforcement . . . 3

4 Dynamic Environmental State Design (`state.c`) **3**
4.1 Algorithmic Optimization in Deck Representations . . . 3
4.2 Bounded vs. Amortized Scale Lists . . . 4

5 Game Evaluation and Mechanistic Interactions **4**
5.1 The Victory Matrix Logic . . . 4
5.2 Alphabetized Output Filtering for Auditing . . . 4

6 Quantitative Testing and Edge Case Mitigation **5**

7 Conclusion **5**

1. Introduction and Architectural Scope

The `spireslayer` executable operates exclusively as a Read-Eval-Print Loop (REPL) engine. Its fundamental responsibility is to ingest natural-language-like commands, determine their grammatical validity against a hard-coded formal specification, and consequentially update an internalized representation of a game environment.

The project demanded heavy compliance with three principal pillars:

1. **Strict Deterministic Parsing:** Ambiguous inputs, trailing garbage characters, or semantically incorrect constructs must categorically trigger an `INVALID` trap rather than a corrupted evaluation.
2. **High-Efficiency State Mappings:** Maintaining entities like individual playing cards, artifacts (relics), and an associative enemy codex without suffering memory bloating or degrading lookup complexity iteratively.
3. **Hardware and Memory Robustness:** Avoiding all undefined C behaviors, strictly catching heap exhaustion limits via safe abstractions mapping `malloc` and `realloc`, and comprehensively terminating execution without dangling pointers.

To meet these requirements, the architecture was segmented into three explicitly decoupled translation units: `main.c` serving as the IO conduit, `parser.c` managing syntax trees and lexical grids, and `state.c` orchestrating the semantic transition sequences.

2. Advanced Lexical Validation

2.1 Raw Stream Processing

The execution cycle triggers a block stream reading up to `MAX_LINE_LENGTH` bytes. Input sanitization constitutes the initial runtime step, where a custom `chomp_newline()` mutator handles mixed OS environments by stripping `\n` and `\r` interchangeably. The hard exit token `"Exit"` bypasses deeper token structures for an instantaneous `strcmp` halt to guarantee performance continuity matching the assignment skeleton.

2.2 Tokenization and Spatial Geometry Tracking

Standard `strtok()` derivations forcefully collapse multiple delimiters. However, our implementation dictated that spatial constraints were structurally significant (e.g., verifying multi-word identifiers had strictly one spacer apart). To accomplish this organically, the payload converts into an array of isolated `Token` structs.

Each token operates not only as a string buffer segment but functionally traces its origin dimension: `orig_start` and `orig_len` map the exact character indexing derived from the unprocessed raw string. This mechanism allows the underlying parser to later mathematically cross-reference token gap geometry without retaining multiple copies of the memory payload.

3. The Backtracking Syntactic Evaluator

Processing game commands such as *"Ironclad buys relic Anchor for 150 gold"* invokes ambiguity concerns for procedural systems. Does the token "for" signal the price block, or does the user's relic inherently possess the name "for"?

3.1 Greedy Matching Strategy

The parser relies on a cascading backtrack resolution sequence. Commands with multi-branch possibilities (e.g., combat scenarios permitting an optional bounty flag) test the longest, most lexically dense evaluation matrix first.

```c
bool parse_fight_command(Token* tokens, int num_tokens, WorldState* state) {
    int saved_pos = pos;
    char* enemy_name = NULL;

    // Attempt the 'Bounty' path utilizing "for" as an exclusion stop-word.
    if (match_name_dynamic(tokens, num_tokens, &pos, &enemy_name, "for")) {
        int initial_bounty_pos = pos;
        if (match_keyword(tokens, "for", &pos) &&
            match_integer(tokens, &bounty_val, &pos) &&
            match_keyword(tokens, "gold", &pos) &&
            pos == num_tokens) {

            handle_fight_bounty(state, enemy_name, bounty_val);
            free(enemy_name);
            return true;
        }
    }

    // Backtrack variables explicitly to nullify corrupted path trace
    pos = saved_pos;
    if (enemy_name) free(enemy_name);

    // Proceed to standard path evaluation...
}
```

Listing 1: Backtracking Strategy for Optional Bounty Grammar

3.2 Semantic Keyword Blacklisting and Contiguity Enforcement

During logical reconstruction of standard nouns (Cards, Relics, Monsters), each token slices against a hardcoded lookup dictionary consisting of 40 specific system-reserved keywords. A noun mapping evaluates to false instantly if any sub-word intercepts a keyword like "fights", "card", or "buys". Subsequently, the positional data harvested during tokenization triggers an offset assertion `(gap != 1)`. The rejection of inputs like `"Twin  Strike"` explicitly highlights our commitment to strict string representation.

4. Dynamic Environmental State Design (`state.c`)

Rather than fracturing variables into disparate globals, an encapsulated pointer array `WorldState` aggregates the context of the user. This enforces thread safety boundaries hypothetically and avoids scoping complexities.

4.1 Algorithmic Optimization in Deck Representations

The standard "Slay the Spire" replication invites immense deck sizes (exceeding hundreds of instances sequentially). If the memory array registered one object per occurrence linearly, scanning mechanisms during search/destroy checks would bottleneck severely. We bypassed this by clustering identities. The `DeckEntry` abstraction possesses structural integer flags: `base_count` and `upgraded_count`. This guarantees that even if a user hoards 40 instances of "Strike", our array size utilizes explicitly one slot segment traversing at $O(1)$ block size scaling efficiency.

4.2 Bounded vs. Amortized Scale Lists

- **Bounded Storage (Potion Belt):** Fixed directly to `POTION_BELT_SIZE` equating strictly to 3. Upon usage, standard right-to-left cascading shifting ensures indexing compaction efficiently without creating memory gaps (null spaces).
- **Unbounded Storage (Codex, Relics):** Operates on amortized doubling metrics. Relying heavily on pointer swaps dynamically verifying `capacity` limit horizons against currently utilized `count`.

```c
// Implementation logic utilized ubiquitously across state allocations
if (state->relic_count >= state->relic_capacity) {
    int new_cap = (state->relic_capacity == 0) ? 8 : state->relic_capacity * 2;
    void* temp_block = realloc(state->relics, new_cap * sizeof(char*));

    // Hardware failure catch to prevent pointer obliteration.
    if (!temp_block) {
        fprintf(stderr, "FATAL: Heap Exhaustion\n");
        exit(EXIT_FAILURE);
    }
    state->relics = temp_block;
    state->relic_capacity = new_cap;
}
```

Listing 2: Defensive Heap Allocation Wrapper

5. Game Evaluation and Mechanistic Interactions

Executing battle configurations relies aggressively on the instantiated `Codex` definitions.

5.1 The Victory Matrix Logic

Whenever combat triggers against a given `Enemy`, the system routes through multi-tier validation checks:

1. Is the enemy indexed within the database? If missing, instant defeat mechanics override.
2. Parse internal associations for `Item Effectiveness`. Match the effective artifacts/cards directly against player's inventory mapping variables.
3. **The Exhaust Tagging Nuance:** The Boolean identifier `exhausts` propagates independently to all matching deck references. On a victorious loop involving marked entities, the core decrements exactly one base copy (falling back defensively to an upgraded trace if bases equal 0).

5.2 Alphabetized Output Filtering for Auditing

Certain interrogative commands dictate strictly alphabetized output feedback. Invoking the baseline `qsort` against our standard structure directly corrupts chronologically relevant queues (specifically, potion placement indexing which dictates inventory dropping operations). Instead, rendering involves generating a surrogate stack-allocated sequence buffer mapping the identical heap pointers, organizing mathematically utilizing native `strcmp` implementations, flushing output directly to stdout buffers, and letting spatial OS scope boundaries implicitly reclaim the index array.

6. Quantitative Testing and Edge Case Mitigation

In combination with a custom procedural Python parsing suite driving 400+ sequential commands tracking literal textual representations, the system is designed hardened against explicitly defined anomalies:

- **Type Overflow Mitigations:** Values read via ‘strtoll‘ explicitly detect 32-bit ‘signed int‘ upper limits, intercepting malicious strings masking as coordinates gracefully. Numbers initiated with explicit 0 representations immediately invoke grammar failure to satisfy strict PDF assignments.
- **Defeat Processing:** Exceeding health reduction parameters strictly clamps the logical integer to ‘0‘ instead of rolling under into negative bounds, retaining continuous executable logic loop iterations natively.
- **String Encapsulation Safety:** Every ‘strdup‘ string generated explicitly associates dynamically with global destruction hooks executed deterministically terminating during the custom ‘EOF‘ or ‘Exit‘ trigger events, asserting total immunity to leaks cleanly detected by GNU ‘AddressSanitizer‘ diagnostics.

7. Conclusion

The framework designed represents a meticulously orchestrated balance between memory sensitivity derived from optimal structural implementations and strict parsing mechanics demanding backtracking versatility. Splitting token-based string validation procedures strictly from underlying associative database functions establishes modular longevity and scalability. *Spireslayer* comfortably traverses extreme parameter limits, executing game environments reliably without compromising processing overheads or semantic accuracy boundaries.

AI Usage Statement

Generative Artificial Intelligence was used very sparingly during this project. Specifically, we utilized AI tools primarily to proofread this document for English grammar and fluency, and to help format a few repetitive inline code comments for better readability. Additionally, we received extremely minor, isolated code assistance strictly for resolving a few trivial syntax warnings and exploring standard C library string function definitions.

The vast majority of the project, including the entirety of the overarching system architecture, core lexical parsing routines, and memory management abstractions, was meticulously designed, implemented, and debugged by hand. Our minimal use of AI strictly respects the unassisted core problem-solving methodologies expected in CMPE 230.